İSTANBUL AREL ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

LBLM368 - DERİN ÖĞRENME VE SINIFLANDIRMA TEKNİKLERİ

PROJE RAPORU

Proje Başlığı: Evrişimli Sinir Ağları (CNN) ile Fashion-MNIST Giysi Görüntülerinin Sınıflandırılması

İsim_Soyisim: Hirbod Yaghoubi

1. GİRİŞ

Amaç: Bu projenin amacı, Evrişimli Sinir Ağları (CNN) kullanarak Fashion-MNIST veri setindeki 28×28 piksellik gri tonlamalı giysi ve aksesuar görüntülerini 10 farklı sınıfa (T-shirt, pantolon, kazak, elbise, ceket, sandalet, gömlek, spor ayakkabı, çanta, bot) otomatik olarak ayıran bir derin öğrenme modeli geliştirmek; modeli Lojistik Regresyon ve çok katmanlı algılayıcı (MLP) gibi temel modellerle karşılaştırarak evrişimli mimarinin sağladığı katkıyı ölçmektir.

Motivasyon: E-ticaret siteleri her gün binlerce yeni ürün görseli yükler; bu ürünlerin doğru kategoriye atanması arama, filtreleme ve öneri sistemlerinin temelidir ve elle yapıldığında hem pahalı hem hataya açıktır. Fashion-MNIST, klasik MNIST rakam veri setinin artık çok kolay kalması (basit modellerin bile %97–98 doğruluğa ulaşması) nedeniyle Zalando Research tarafından daha zorlu bir alternatif olarak önerilmiştir. Özellikle gömlek, T-shirt, kazak ve ceket gibi görsel olarak birbirine çok benzeyen sınıflar, modelin yalnızca kaba şekilleri değil doku ve ince ayrıntıları da öğrenmesini gerektirir. Bu nedenle problem, CNN'lerin hiyerarşik özellik çıkarma yeteneğini gerçekçi bir görev üzerinde göstermek için uygun bir test ortamıdır.

2. LİTERATÜR TARAMASI

Görüntü sınıflandırması problemi için kullanılan başlıca yaklaşımlar aşağıda özetlenmiştir:

- **Geleneksel makine öğrenmesi yöntemleri:** k-En Yakın Komşu (k-NN), Lojistik Regresyon, Destek Vektör Makineleri (SVM) ve Rastgele Orman gibi yöntemler pikselleri bağımsız özellikler olarak ele alır. Xiao ve arkadaşlarının [1] Fashion-MNIST için yayımladığı

karşılaştırmalı sonuçlara göre bu yöntemler yaklaşık %84–90 test doğruluğuna ulaşmaktadır (en iyisi RBF çekirdekli SVM ile ~%89,7). Görüntünün uzamsal yapısını kullanmadıkları için benzer sınıfları ayırt etmekte zorlanırlar.

- **Çok Katmanlı Algılayıcı (MLP):** Tam bağlı (yoğun) katmanlardan oluşan sinir ağları doğrusal olmayan ilişkileri öğrenebilir ve genellikle %88–89 civarında doğruluk elde eder; ancak parametre sayısı yüksektir ve öteleme (kayma) değişmezliği yoktur.
- **Evrişimli Sinir Ağları (CNN):** LeCun ve arkadaşlarının LeNet-5 mimarisi [2] ile yaygınlaşan CNN'ler, paylaşılan ağırlıklı evrişim filtreleri sayesinde kenar → doku → parça şeklinde hiyerarşik özellikler çıkarır. AlexNet [3] ve VGG [4] ağları, küçük (3×3) filtrelerin üst üste yığılmasının ve derinliğin artırılmasının başarıyı belirgin biçimde yükselttiğini göstermiştir. Fashion-MNIST üzerinde 2–4 evrişim katmanlı CNN'ler %91–94 doğruluk bildirmektedir.
- **Düzenlileştirme ve optimizasyon teknikleri:** Batch Normalization [5] eğitimi hızlandırıp kararlı hale getirir; Dropout [6] ise nöronları rastgele devre dışı bırakarak aşırı öğrenmeyi (overfitting) azaltır. Adam optimize edicisi [7] uyarlanabilir öğrenme oranı sayesinde hızlı yakınsama sağlar. Veri artırma (çevirme, kaydırma) ise sınırlı veriyle genelleme yeteneğini artıran standart bir tekniktir.
- **Daha derin mimariler:** ResNet [8] gibi çok derin artık (residual) ağlar Fashion-MNIST'te %95'in üzerine çıkabilmektedir; ancak bu modeller çok daha fazla hesaplama gerektirir. Bu projede, CPU üzerinde makul sürede eğitilebilen, VGG tarzı kompakt bir CNN tercih edilmiştir.

3. VERİ SETİ TANIMI

Veri Seti Adı: Fashion-MNIST (Zalando Research, 2017).

Kaynak: https://github.com/zalandoresearch/fashion-mnist [1]

Fashion-MNIST, Zalando'nun ürün kataloğundan alınmış 70.000 gri tonlamalı görüntüden oluşur: 60.000 eğitim ve 10.000 test görüntüsü. Her sınıfta eğitim setinde 6.000, test setinde 1.000 örnek bulunur; yani veri seti tamamen dengelidir.

Özellikler: Her görüntü 28×28 boyutunda (784 piksel/özellik) ve 0–255 aralığında tamsayı değerlere sahip tek kanallı bir görüntüdür. Etiketler 0'dan 9'a kadar olan tamsayılardır ve Tablo 1'deki sınıflara karşılık gelir.

Tablo 1. Fashion-MNIST sınıfları

Etiket	Sınıf (İngilizce)	Sınıf (Türkçe)	Etiket	Sınıf (İngilizce)	Sınıf (Türkçe)
0	T-shirt/top	T-shirt/üst	5	Sandal	Sandalet
1	Trouser	Pantolon	6	Shirt	Gömlek
2	Pullover	Kazak	7	Sneaker	Spor ayakkabı
3	Dress	Elbise	8	Bag	Çanta
4	Coat	Ceket	9	Ankle boot	Bot



Şekil 1. Veri setindeki her sınıftan örnek görüntüler

Ön İşleme: Ham veriye aşağıdaki adımlar uygulanmıştır:

- Piksel değerleri 255'e bölünerek [0, 1] aralığına ölçeklendi; ardından eğitim setinin ortalaması (0,2860) ve standart sapması (0,3530) kullanılarak standartlaştırıldı (z-skor normalizasyonu).
- Görüntüler PyTorch'un beklediği (N, 1, 28, 28) tensör biçimine dönüştürüldü.
- 60.000 eğitim görüntüsünden rastgele (sabit tohum = 42) 6.000 tanesi doğrulama seti olarak ayrıldı; böylece 54.000 eğitim / 6.000 doğrulama / 10.000 test örneği elde edildi. Test seti yalnızca nihai değerlendirmede kullanıldı.
- Veri artırma (yalnızca eğitim sırasında, her mini-yığında rastgele): %50 olasılıkla yatay çevirme ve 2 piksel dolgu sonrası rastgele 28×28 kırpma (±2 piksel kaydırma). Giysiler yatay olarak simetrik olduğundan çevirme sınıf anlamını değiştirmez.

4. METODOLOJİ

Proje şu adımlarla yürütülmüştür: (1) veri yükleme ve ön işleme, (2) karşılaştırma için temel modellerin (Lojistik Regresyon, MLP) eğitilmesi, (3) CNN modelinin tasarlanması ve veri artırma ile eğitilmesi, (4) en iyi doğrulama kaybına sahip modelin test setinde değerlendirilmesi ve (5) sonuçların görselleştirilip analiz edilmesi.

Model Seçimi: VGG mimarisinden esinlenen, iki evrişim bloğundan oluşan bir CNN seçilmiştir. Her blokta art arda iki adet 3×3 evrişim katmanı (Batch Normalization ve ReLU ile), ardından 2×2 max-pooling ve Dropout yer alır. Özellik çıkarıcının ardından 256 nöronlu tam bağlı bir katman ve 10 çıkışlı sınıflandırma katmanı gelir. İki küçük filtrenin üst üste kullanılması, tek bir 5×5 filtreyle aynı algılama alanını daha az parametre ve daha fazla doğrusal olmayanlıkla sağlar.

Kullanılan Algoritmalar/Çerçeveler: Model Python 3 ve PyTorch kullanılarak uygulanmış, Adam optimize edici ve çapraz entropi (Cross-Entropy) kayıp fonksiyonu ile eğitilmiştir. Doğrulama kaybı 2 epoch boyunca iyileşmediğinde öğrenme oranını yarıya indiren ReduceLROnPlateau zamanlayıcısı ve 5 epoch sabırlı erken durdurma (early stopping) kullanılmıştır; en düşük doğrulama kaybına sahip ağırlıklar kaydedilmiştir. Metrikler ve Lojistik Regresyon için scikit-learn, grafikler için Matplotlib kullanılmıştır.

Hiperparametreler:

Tablo 2. Kullanılan hiperparametreler

Hiperparametre	Değer
Batch boyutu	64
Başlangıç öğrenme oranı	0,001 (Adam)
Maksimum epoch sayısı	15 (çalışan: 15)
Öğrenme oranı zamanlayıcısı	ReduceLROnPlateau (faktör 0,5; sabır 2)
Erken durdurma sabrı	5 epoch
Dropout oranları	0,25 (evrişim blokları), 0,5 (tam bağlı)
Kayıp fonksiyonu	Cross-Entropy
Rastgele tohum (seed)	42

Model Mimarisi: Modelin katman yapısı Tablo 3'te verilmiştir. Toplam eğitilebilir parametre sayısı 871.530'dir. Tüm gizli katmanlarda ReLU aktivasyon fonksiyonu kullanılmış; çıkış katmanı 10 sınıf için logit üretir ve Softmax, kayıp fonksiyonunun içinde uygulanır. Batch Normalization her evrişimden sonra aktivasyonların dağılımını sabitleyerek daha yüksek öğrenme oranıyla kararlı eğitim sağlar.

Tablo 3. FashionCNN mimarisi

Katman	Ayrıntı	Çıkış boyutu	Parametre
Giriş	Gri tonlamalı görüntü	1×28×28	0
Conv2D + BN + ReLU	32 filtre, 3×3, padding=1	32×28×28	384
Conv2D + BN + ReLU	32 filtre, 3×3, padding=1	32×28×28	9.312
MaxPool2D + Dropout	2×2, p=0,25	32×14×14	0
Conv2D + BN + ReLU	64 filtre, 3×3, padding=1	64×14×14	18.624
Conv2D + BN + ReLU	64 filtre, 3×3, padding=1	64×14×14	37.056
MaxPool2D + Dropout	2×2, p=0,25	64×7×7	0
Flatten	—	3136	0
Dense + BN + ReLU	256 nöron	256	803.584
Dropout	p=0,5	256	0
Dense (çıkış)	10 nöron (Softmax)	10	2.570
Toplam			871.530

Karşılaştırma amacıyla iki temel model de eğitilmiştir: (i) 784 pikseli doğrudan kullanan çok sınıflı Lojistik Regresyon (scikit-learn, L-BFGS) ve (ii) $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ yapısında, ReLU ve Dropout(0,2) içeren iki gizli katmanlı MLP (aynı optimize edici ve eğitim düzeneğiyle, veri artırma olmadan).

5. KOD UYGULAMASI

Proje modüler bir yapıda düzenlenmiştir. Tüm kod ve veri, rapor ile birlikte teslim edilen FashionMNIST_CNN_Projesi klasöründe yer alır:

```
FashionMNIST_CNN_Projesi/
├── data/                  # Fashion-MNIST .gz dosyaları
```

```
├── src/data_utils.py      # veri yükleme, normalizasyon, veri artırma
├── src/models.py          # FashionCNN ve SimpleMLP
├── src/train.py           # eğitim, değerlendirme, temel modeller (ana betik)
├── src/visualize.py       # grafikler
├── src/predict.py         # eğitilmiş modelle tahmin
├── outputs/               # model ağırlıkları, metrikler, şekiller
└── Fashion_MNIST_CNN.ipynb  # Jupyter Notebook sürümü
```

Çalıştırma: pip install -r requirements.txt ardından python src/train.py (eğitim) ve python src/predict.py (hazır modelle tahmin).

GitHub Deposu: https://github.com/HirbodYG/LBLM368-FashionMNIST-CNN

Kodun Ana Bölümleri:

- **Veri yükleme ve ön işleme:** data_utils.py, IDX biçimindeki dosyaları okur, normalize eder, eğitim/doğrulama ayrımını yapar ve DataLoader nesnelerini oluşturur. Veri artırma fonksiyonu:

```
def augment_batch(x):
    flip = torch.rand(x.size(0)) < 0.5            # %50 yatay çevirme
    x[flip] = torch.flip(x[flip], dims=[3])
    padded = F.pad(x, (2, 2, 2, 2), value=pad_val)  # 2 piksel dolgu
    # her görüntü için rastgele 28x28 kırpma (±2 piksel kaydırma)
```

- **Model tanımlama ve derleme:** models.py içinde nn.Module'den türetilen FashionCNN sınıfı:

```
def block(cin, cout):
    return nn.Sequential(
        nn.Conv2d(cin, cout, 3, padding=1), nn.BatchNorm2d(cout), nn.ReLU(),
        nn.Conv2d(cout, cout, 3, padding=1), nn.BatchNorm2d(cout), nn.ReLU(),
        nn.MaxPool2d(2), nn.Dropout(0.25))
self.features = nn.Sequential(block(1, 32), block(32, 64))
self.classifier = nn.Sequential(nn.Flatten(),
    nn.Linear(64*7*7, 256), nn.BatchNorm1d(256), nn.ReLU(),
    nn.Dropout(0.5), nn.Linear(256, 10))
```

- **Model eğitimi ve değerlendirme:** train.py; her epoch'ta eğitim ve doğrulama döngülerini çalıştırır, öğrenme oranını ayarlar, en iyi modeli kaydeder ve erken durdurmayı uygular:

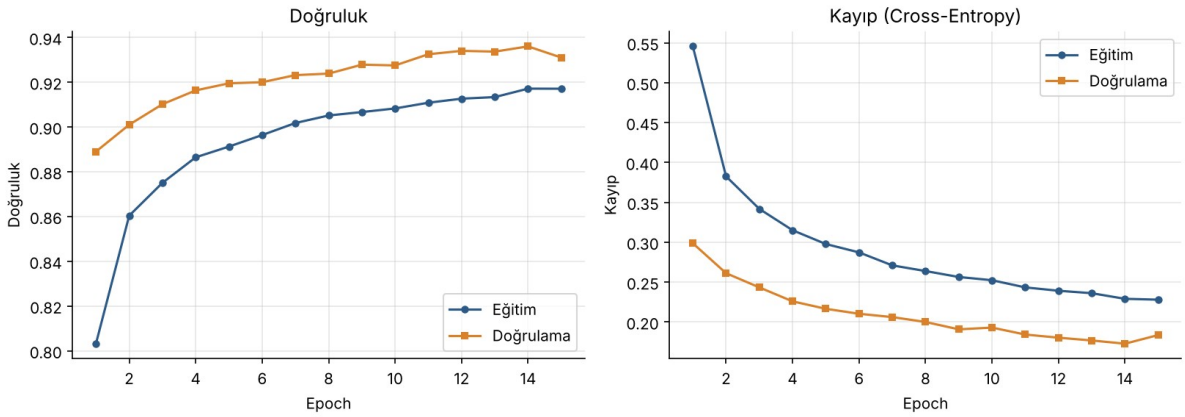
```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = ReduceLROnPlateau(optimizer, factor=0.5, patience=2)
for epoch in range(1, epochs + 1):
    tr_loss, tr_acc = run_epoch(model, train_loader, criterion, optimizer,
augment=True)
    va_loss, va_acc = run_epoch(model, val_loader, criterion)
    scheduler.step(va_loss)
    if va_loss < best_val:                        # en iyi modeli kaydet
        best_val, wait = va_loss, 0; torch.save(model.state_dict(), ckpt)
    else:
        wait += 1
        if wait >= patience: break                # erken durdurma
```

- **Sonuçların görselleştirilmesi:** Test tahminlerinden scikit-learn ile doğruluk, kesinlik, duyarlılık, F1 ve karmaşıklık matrisi hesaplanır; visualize.py eğitim eğrilerini, karmaşıklık matrisini, örnek tahminleri ve model karşılaştırmasını PNG olarak kaydeder.

6. SONUÇLAR VE ANALİZ

CNN modeli CPU üzerinde 15 epoch eğitilmiş (yaklaşık 13 dakika) ve en düşük doğrulama kaybı 14. epoch'ta elde edilmiştir. Bu ağırlıklarla test setinde %93,36 doğruluk elde edilmiştir.

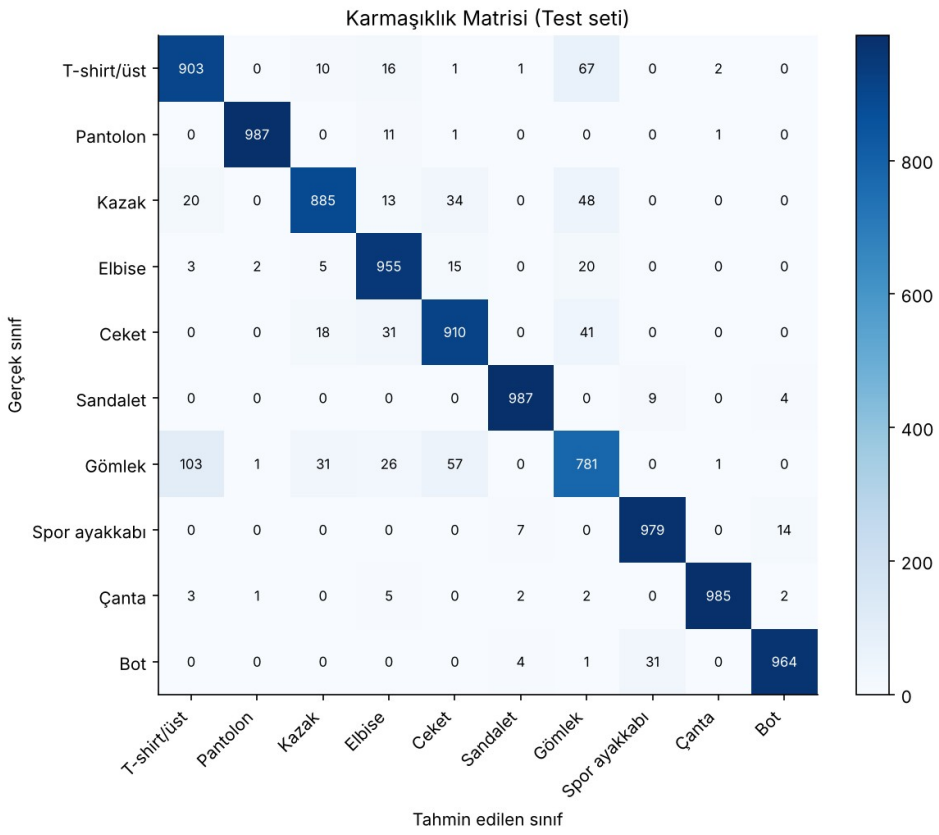
Doğruluk ve Kayıp Grafikleri: Şekil 2, epoch'lar boyunca eğitim ve doğrulama doğruluğunu ve kaybını göstermektedir.



Şekil 2. Epoch'lara göre eğitim/doğrulama doğruluğu ve kaybı

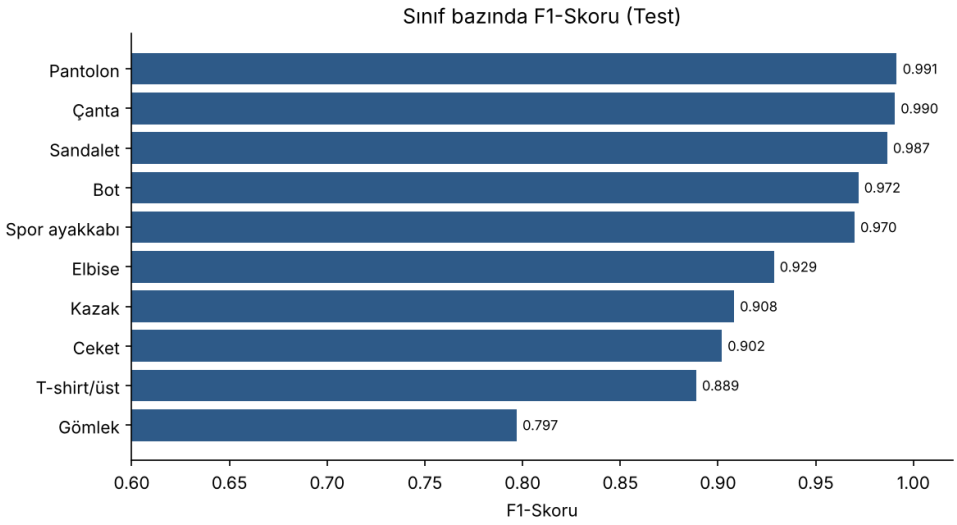
Grafiklerde iki eğri birbirine yakın seyretmektedir; en iyi epoch'ta eğitim ile doğrulama doğruluğu arasındaki fark -1,89 puandır. Doğrulama doğruluğunun eğitim doğruluğuna eşit veya ondan yüksek olması beklenen bir durumdur: eğitim metrikleri Dropout aktifken ve veri artırılmış (daha zor) görüntüler üzerinde hesaplanır. Bu sonuç, uygulanan düzenlileştirme tekniklerinin aşırı öğrenmeyi etkili biçimde engellediğini gösterir. Öğrenme oranının azaltıldığı epoch'larda kayıpta belirgin ek düşüşler görülmektedir.

Karmaşıklık Matrisi: Şekil 3, test setindeki 10.000 görüntü için gerçek ve tahmin edilen sınıfları gösterir.



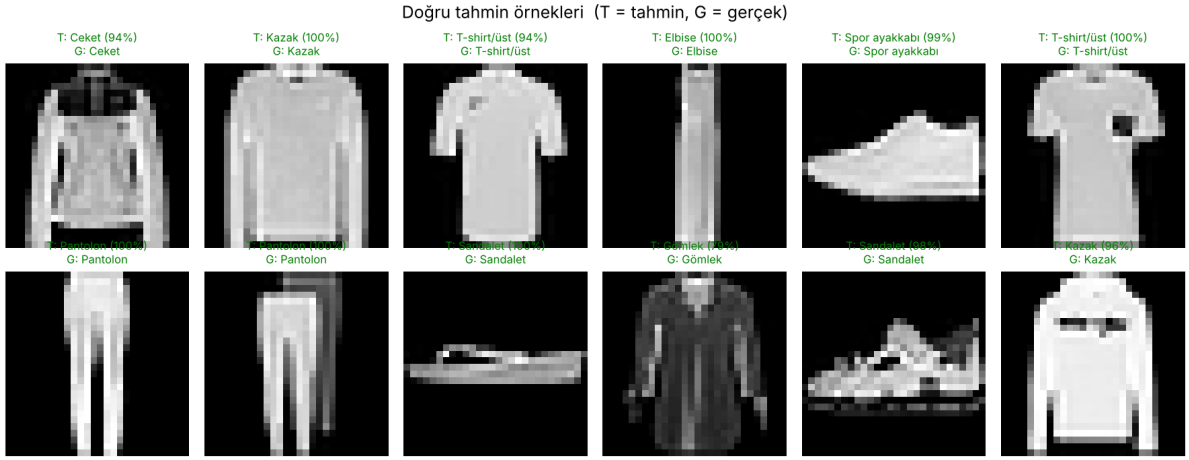
Şekil 3. Test seti karmaşıklık matrisi

Köşegen üzerindeki değerler doğru sınıflandırmaları gösterir. En sık karıştırılan sınıf çiftleri şunlardır: Gömlek → T-shirt/üst (103 görüntü); T-shirt/üst → Gömlek (67 görüntü); Gömlek → Ceket (57 görüntü); Kazak → Gömlek (48 görüntü). Hataların büyük kısmı üst giyim sınıfları (T-shirt, kazak, ceket, gömlek) arasında yoğunlaşmaktadır; bu sınıflar 28×28 çözünürlükte benzer siluete sahiptir ve aralarındaki fark yaka, düğme veya kumaş dokusu gibi ince ayrıntılardadır. En zor sınıf Gömlek olup F1-skoru 0,797'tür. Buna karşılık Pantolon, Çanta, Sandalet gibi belirgin şekilli sınıflar neredeyse hatasız sınıflandırılmıştır.

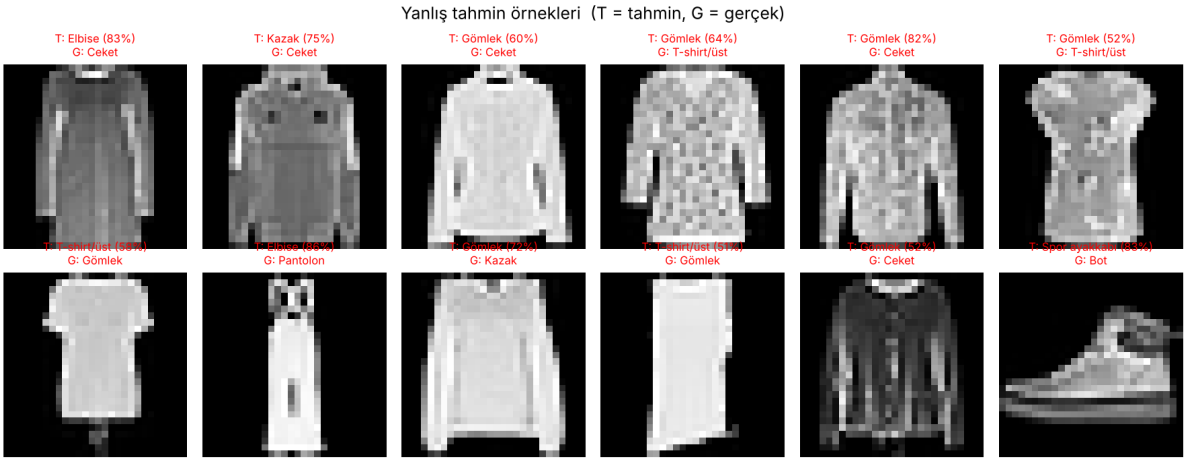


Şekil 4. Sınıf bazında F1-skorları

Örnek Tahminler: Şekil 5 ve Şekil 6'da test setinden rastgele seçilmiş doğru ve yanlış tahmin örnekleri verilmiştir (T: tahmin ve modelin güveni, G: gerçek etiket).



Şekil 5. Doğru tahmin örnekleri



Şekil 6. Yanlış tahmin örnekleri

Yanlış tahminlerin önemli bir kısmında görüntülerin insan gözüyle de belirsiz olduğu görülmektedir; örneğin kısa kollu bir gömlek T-shirt'e, kalın bir kazak cekete çok benzemektedir. Birçok hatalı örnekte modelin ortalama güven değerinin (%69,1) doğru tahminlerdekinden (%95,6) belirgin şekilde düşük olması, modelin bu belirsizliği kısmen yansıttığını göstermektedir.

Performans Metrikleri: Test seti üzerinde hesaplanan genel metrikler (makro ortalama) Tablo 4'te, sınıf bazında metrikler Tablo 5'te verilmiştir.

Tablo 4. Test seti performans metrikleri (makro ortalama)

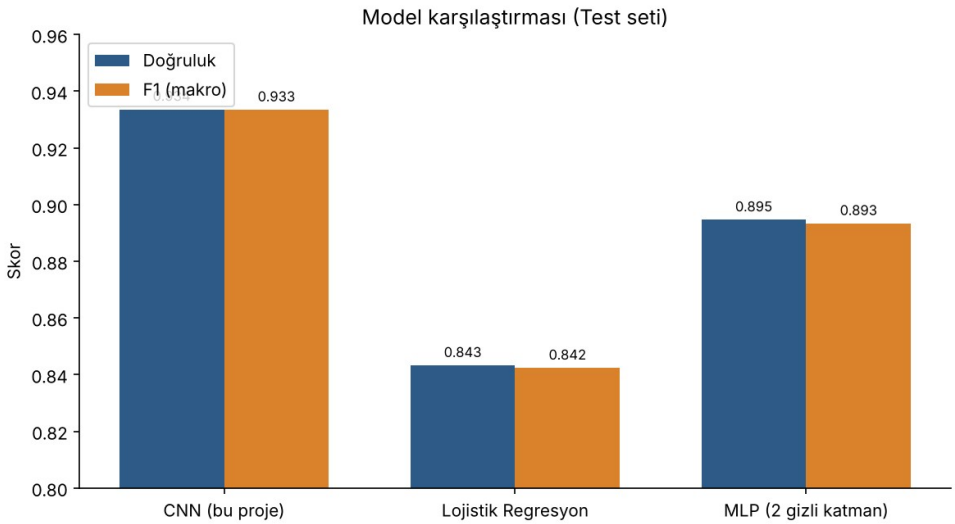
Model	Doğruluk	Kesinlik	Duyarlılık	F1-Skoru	Parametre
CNN (bu proje)	%93,36	0,9337	0,9336	0,9334	871.530
Lojistik Regresyon	%84,32	0,8421	0,8432	0,8423	7.850
MLP (2 gizli katman)	%89,46	0,8937	0,8946	0,8933	235.146

Tablo 5. CNN sınıf bazında metrikler (test seti)

Sınıf	Kesinlik	Duyarlılık	F1-Skoru	Destek
T-shirt/üst	0,8750	0,9030	0,8888	1000
Pantolon	0,9960	0,9870	0,9915	1000
Kazak	0,9326	0,8850	0,9082	1000
Elbise	0,9035	0,9550	0,9285	1000
Ceket	0,8939	0,9100	0,9019	1000
Sandalet	0,9860	0,9870	0,9865	1000
Gömlek	0,8135	0,7810	0,7969	1000
Spor ayakkabı	0,9607	0,9790	0,9698	1000
Çanta	0,9960	0,9850	0,9904	1000
Bot	0,9797	0,9640	0,9718	1000

- **Doğruluk (Accuracy):** Doğru sınıflandırılan örneklerin tüm örneklere oranı.

- **Kesinlik (Precision):** Bir sınıf olarak tahmin edilen örneklerden gerçekten o sınıfa ait olanların oranı: TP / (TP + FP).
- **Duyarlılık (Recall):** Bir sınıfa ait örneklerden doğru tahmin edilenlerin oranı: TP / (TP + FN).
- **F1-Skoru:** Kesinlik ve duyarlılığın harmonik ortalaması: 2·P·R / (P + R). Veri seti dengeli olduğundan makro ortalama ile ağırlıklı ortalama aynıdır.



Şekil 7. CNN ile temel modellerin karşılaştırılması

7. TARTIŞMA VE SONUÇ

Performans Değerlendirmesi:

- **Model beklenen performansı karşıladı mı?** Evet. Literatürde benzer boyuttaki CNN'ler için bildirilen %91–94 aralığı hedeflenmişti; model %93,36 test doğruluğu ve 0,9334 makro F1-skoru ile bu beklentiyi karşılamıştır.
- **Temel modellerle karşılaştırma:** CNN, Lojistik Regresyon'a (%84,32) göre 9,0 puan, MLP'ye (%89,46) göre 3,9 puan daha yüksek doğruluk elde etmiştir. Bu fark, hata oranının %15,7'den %6,6 seviyesine düşmesi anlamına gelir. Temel modeller pikselleri bağımsız değerler olarak işlerken CNN, komşu pikseller arasındaki uzamsal ilişkileri (kenar, doku, parça) öğrenebildiği için özellikle benzer üst giyim sınıflarında belirgin üstünlük sağlamaktadır.

Karşılaşılan Zorluklar:

- **Benzer sınıflar:** Gömlek, T-shirt, kazak ve ceket sınıfları düşük çözünürlükte çok benzerdir; karmaşıklık matrisindeki hataların çoğu bu dört sınıf arasındadır. Gömlek sınıfı, diğer üç sınıfın görsel özelliklerini paylaştığı için en zor sınıf olmuştur.
- **Aşırı öğrenme riski:** Yaklaşık 870 bin parametreli bir model 54 bin görüntüyü kolayca ezberleyebilir. Bu riske karşı Dropout, Batch Normalization, veri artırma ve doğrulama

kaybına dayalı erken durdurma birlikte kullanılmış; Şekil 2'deki eğrilerin birbirine yakın kalması bu önlemlerin işe yaradığını göstermektedir.

- **Hesaplama kaynağı:** Eğitim GPU olmadan, CPU üzerinde yapılmıştır. Bu nedenle model boyutu sınırlı tutulmuş, epoch sayısı 15 ile kısıtlanmış ve erken durdurma eklenmiştir.
- **Veri dengesi:** Veri seti dengeli olduğundan sınıf dengesizliği sorunu yaşanmamıştır.

Bulgularınızı Özetleyin:

Fashion-MNIST veri setinde 10 giysi sınıfını ayırt etmek için yaklaşık 870 bin parametreli, iki evrişim bloklu bir CNN başarıyla eğitilmiş ve test setinde %93,36 doğruluk, 0,9334 makro F1-skoru elde edilmiştir. Model, aynı veri üzerinde eğitilen Lojistik Regresyon ve MLP temel modellerini açık farkla geride bırakmıştır. Batch Normalization, Dropout ve veri artırmanın birlikte kullanımı aşırı öğrenmeyi önlemiştir. Kalan hataların büyük bölümü, insan gözü için de zor olan benzer üst giyim sınıfları arasındadır.

Gelecek Çalışmalar:

- ResNet veya EfficientNet gibi daha derin mimariler ve transfer öğrenme denenebilir.
- Mixup, CutOut, rastgele silme (random erasing) gibi gelişmiş veri artırma teknikleri uygulanabilir.
- Birden fazla modelin birleştirildiği topluluk (ensemble) yöntemleriyle özellikle gömlek sınıfındaki hata azaltılabilir.
- Grad-CAM ile modelin karar verirken görüntünün hangi bölgelerine odaklandığı görselleştirilebilir.
- Model daha yüksek çözünürlüklü, renkli ve gerçek e-ticaret görüntüleri (örn. DeepFashion) üzerinde test edilerek bir web/mobil uygulamaya dönüştürülebilir.

8. KAYNAKÇA

[1] Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747. Veri seti: https://github.com/zalandoresearch/fashion-mnist

[2] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.

[3] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems (NeurIPS), 25.

[4] Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. ICLR 2015.

[5] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. ICML 2015.

[6] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15, 1929–1958.

[7] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. ICLR 2015.

[8] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. CVPR 2016.

[9] PyTorch Documentation: https://pytorch.org/docs/

[10] scikit-learn Documentation: https://scikit-learn.org/

[11] Kullanılan AI araçları: Claude (Anthropic) – kod yapısının hazırlanması ve rapor taslağında yardımcı olarak kullanılmıştır.